Algorithm 2 Vanilla Progress and Compress Algorithm

Initialize: policy networks π_{active}, π_{kb}; encoders ϕ_{active}, ϕ_{kb}; tasks $\mathcal{T}$; timesteps $T_{active}, T_{kb}, T_{Fisher}$; buffer U; encoder f; visits $\mathcal{V}$
for each visit v in $\mathcal{V}$ **do**
  for each task k in $\mathcal{T}$ **do**
    Set up environment E^k
    Unfreeze $\pi_{active}, \phi_{active}, f$; Freeze π_{kb}, ϕ_{kb}
    for $t = 1$ to T_{active} **do** ▷ Progress Phase
      Observe $s_t \sim E^k$, $\phi = \phi_{active}(s_t)$, $a_t \sim \pi_{active}(f(\phi))$
      $r_{t+1}, s_{t+1} \sim P^{E^k}(s_t, a_t)$, $U \leftarrow U \cup \{(r_{t+1}, s_t, s_{t+1}, a_t)\}$
      if $t \mod \text{size}(U) = 0$ **then**
        Evaluate actions based on rollout U
        Update θ_{active} using SGD; Clear U ▷ the loss $\mathcal{L}^{progress}(\theta_{active})$
      end if
    end for
    Switch modes: Freeze $\pi_{active}, \phi_{active}$; Unfreeze π_{kb}, ϕ_{kb}
    for $t = 1$ to T_{kb} **do** ▷ Compress Phase
      Observe $s_t \sim E^k$, $\phi = \phi_{kb}(s_t)$, $a_t \sim \pi_{kb}(\phi)$
      $s_{t+1} \sim P^{E^k}(s_t, a_t)$, $U \leftarrow U \cup \{(s_t, s_{t+1}, a_t)\}$
      if $t \mod \text{size}(U) = 0$ **then**
        if $k < 2$ **then**
          Update θ_{kb} with SGD on $D_{\mathrm{KL}}(\pi_{active}(\cdot|s_{t:\text{size}(U)}) \parallel \pi_{kb}(\cdot|s_{t:\text{size}(U)}))$
        else
          Update θ_{kb} with SGD on $D_{\mathrm{KL}}(\pi_{active}(\cdot|s_{t:\text{size}(U)}) \parallel \pi_{kb}(\cdot|s_{t:\text{size}(U)}))$
$+\frac{\lambda}{2} F_{k-1} \left\| \theta_{kb} - \theta_{kb}^{*(k-1)} \right\|_2^2$
        end if
        Clear U
      end if
    end for
    for $t = 1$ to T_{Fisher} **do**
      Observe $s_t \sim E^k$, $\phi = \phi_{kb}(s_t)$, $a_t \sim \pi_{kb}(\phi)$, $s_{t+1} \sim P^{E^k}(s_t, a_t)$
      $U \leftarrow U \cup \{(s_t, s_{t+1}, a_t)\}$
      if $t \mod \text{size}(U) = 0$ **then**
        Estimate Fisher information based on U; Clear U
      end if
    end for
    Update Fisher information for θ_{kb}^*; Reinitialize θ_{active} ▷ see Equation A2
  end for
end for

Appendix B Architecture Details

B.1 A2C Hyperparameters

The following hyperparameters (see Table B1) specify both the active column and knowledge base networks, which are used in both progress and compress phases for representing the actor (policy) and critic (value function).

Table B1: Neural network architecture of the active column and knowledge base

Order	Layer Type	Activation	Size	Filter Size	Filter Stride
1	Convolution	ReLU	32	[8 × 8]	[4 × 4]
2	Convolution	ReLU	64	[4 × 4]	[2 × 2]
3	Convolution	ReLU	32	[3 × 3]	[1 × 1]
4	Flatten	-	-	-	-
5	Fully-Connected	ReLU	512 Neurons	-	-
6	Critic	Linear	1 Neuron	-	-
7	Actor(Policy)	Linear	4 Neurons	-	-

B.2 Forward Model Hyperparameters

The following hyperparameters (see Table B2) are utilized to assemble the forward model incorporated in the implementation. In this process, the action is one-hot encoded and subsequently merged with an encoded state feature representation.

Table B2: Neural network architecture of the forward model

Order	Layer Type	Activation	Size	Filter Size	Filter Stride
1	Convolution	ReLU	32	[3 × 3]	[2 × 2]
2	Convolution	ReLU	32	[3 × 3]	[2 × 2]
3	Convolution	ReLU	32	[3 × 3]	[2 × 2]
4	Convolution	ReLU	32	[3 × 3]	[2 × 2]
5	Convolution	ReLU	32	[3 × 3]	[2 × 2]
6	Fully-Connected	ReLU	256 Neurons	-	-
7	Fully-Connected	Linear	288 Neurons	-	-

B.3 Hyperparameters of the Experiments

All variations in timesteps across experiments are explicitly stated. One important parameter is the "num-samples-drawn-in-task-agnostic-phase" (see Table B3). This represents a procedure in the task-agnostic phase where games are randomly selected and trained for a specific duration set by "num-env-steps-agnostic". Then, the compress phase is executed for "num-env-steps-compress-agnostic". This process is repeated 30 times, where 30 samples are uniformly drawn from the Meta-Environment. The Meta-Environment includes Pong (P), SpaceInvaders (S), and BeamRider (B). The RMSprop optimizer was consistently used in all experiments.

Table B3: Hyperparameter values in the Progress & Compress baseline, TAPD, Progressive Nets, and Online EWC of the task-agnostic phase

Parameter	**Baseline**	**Task-Agnostic Policy Distillation (TAPD)**	**Progressive Nets**	**Online EWC**
agnostic-phase	-	True	-	-
batch-size-fisher (Fisher information estimation)	32			
eval-steps	10^5			
ewc-lambda	2			
ewc-gamma	0.3			
gamma (Discount factor for rewards)	0.99			
ewc-start	15×10^4			
entropy-coef (Entropy term coefficient)	0.01			
lr (Learning rate)	7×10^{-4}			
eps (RMSprop optimizer epsilon)	1×10^{-5}			
alpha (RMSprop optimizer alpha)	0.99			
num-env-steps-agnostic	-	3×10^5	-	-
num-env-steps-compress (T_{kb})	3×10^5		-	-
num-env-steps-agnostic-compress	-	3×10^5	-	-
num-env-steps-progress (T_{active})	2.5×10^6			
num-processes	10			
num-visits	3			
value-loss-coef	0.5			
max-grad-norm	0.5			
num-steps-fisher (T_{Fisher}) [11]	100			
num-steps (Rollout size)	20			
num-samples-drawn-in-task-agnostic-phase	-	25	-	-
Tasks in task-agnostic phase	-	S, B	-	-
Tasks in progress and compress phases / Training phase	Pong (P), SpaceInvaders (S), BeamRider (B), DemonAttack (D), AirRaid (A)			

References

[1] Speranza, L., Di Porzio, U., Viggiano, D., Donato, A., Volpicelli, F.: Dopamine: The neuromodulator of long-term synaptic plasticity, reward and movement control. Cells **10**(4), 735 (2021)

[2] Sutton, R.S., Barto, A.G.: Reinforcement learning: An introduction (2018)

[3] Miller, K.D., Keller, J.B., Stryker, M.P.: Ocular dominance column development: analysis and simulation. Science **245**(4918), 605–615 (1989)